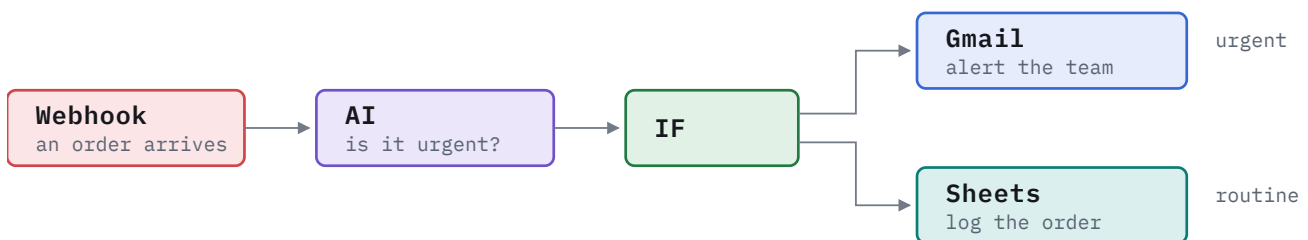


We are going to build n8n. Here is how.

An AI automation platform, in six phases. This deck explains what each phase is, why it exists, and how we will know it is finished — one page per phase, with a picture.

6 PHASES **15** STAGES **13** CORE NODES **5** OF US **Week 14** DEMO **Week 15** REPORT

WHAT IT LOOKS LIKE WHEN WE ARE FINISHED



THE WHOLE PROJECT ON ONE LINE

- 1 Make it run.** A script that executes a workflow. No web page yet.
- 2 Make it real.** Save workflows in a database; draw them on a canvas.
- 3 Make it usable.** Settings forms that build themselves, and `{{ }}` expressions.
- 4 Make it start by itself.** Webhooks and schedules.
- 5 Make it do things.** Google login, the four Google nodes, Code, AI.
- 6 Ship it.** Put it online, measure it against n8n, write the report.

We are building the thing we have been using

Remember our weather workflow. When we pressed **Execute**, n8n did exactly four things — and those four things are the entire project.

1. **It read a drawing.** A canvas is stored as JSON: a list of boxes and a list of lines. That is all a workflow ever is.
2. **It found the starting box** and handed it an empty piece of data.
3. **It ran each box in turn**, passing what came out of one into the next.
4. **It wrote down what happened**, so you could click a node and see its data.



Figure 1. The same workflow, twice. Our canvas produces the JSON; our engine reads it. Nothing else passes between them — which is why two of us can build the canvas and the engine at the same time.

SAY THIS OUT LOUD

“Workflow automation platform” sounds enormous. The core is: read some JSON, loop over it, call some functions. Everything else in this deck is a database, a web page and some API calls arranged around that loop.

Everything between nodes is a *list*

If we get this wrong, we rewrite every node in Week 9. So we decide it now, copy it from n8n deliberately, and never change it.



Figure 2. The Gmail node runs *once*, receives all 50 items, and loops over them itself. The engine never loops on a node's behalf.

IDEA 1

A node is just a function

Input and settings go in, output comes out. The icon and the sidebar form are decoration around that.

IDEA 2

Data is always a list

Every node takes a list of items and returns a list. That is what makes "50 rows → 50 emails" work with no special code.

IDEA 3

Forms build themselves

A node *describes* its settings as data; one piece of React turns any description into a form. Built once, in phase 3.

THE BUG WE WILL ALL HIT

Returning `[items]` where we meant `[[items]]`, or the other way round. The outer array is the list of *output branches*; the inner array is the items on one branch. A node with one output returns `[items]`. An IF node returns `[trueItems, falseItems]`.

How the engine actually runs a workflow

It keeps a **ready list** of nodes whose input has arrived. Take one off, run it, put whatever it feeds onto the list. When the list is empty, the run is over.

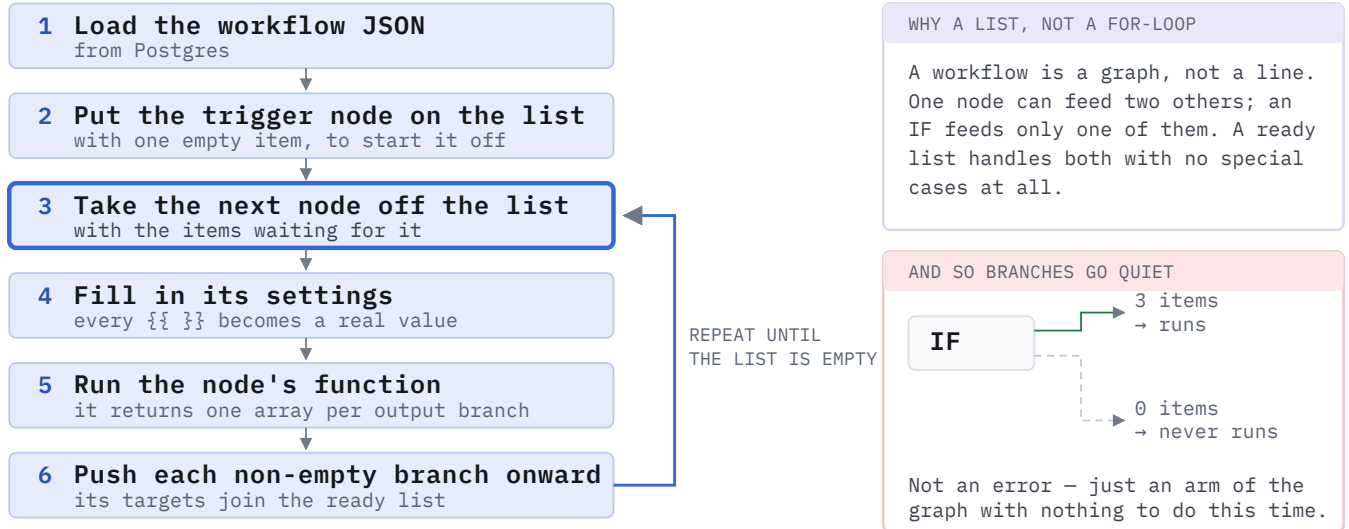


Figure 3. Steps 3 to 6 are about forty lines of JavaScript. Every other phase in this deck exists to feed this loop or to show its results.

SAY THIS OUT LOUD

This is the whole product. If all five of us can explain this diagram, the project is already mostly de-risked — everything after it is work we know how to do.

Six phases, twelve weeks

Each phase ends in something we can actually run and show. We never work on two phases at once, and we never start a phase before the previous one's **“done when”** is true.

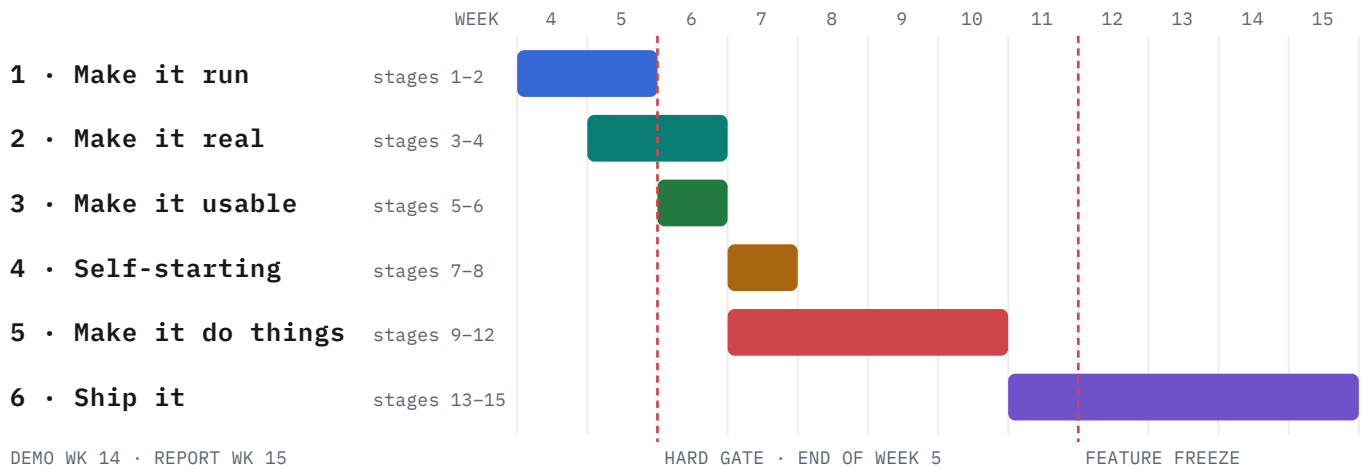


Figure 4. Two dates are non-negotiable. By the end of Week 5 a workflow must run end to end, however ugly. At the end of Week 11 we stop adding features — after that it is only bug fixing, measuring and writing.

IF WE FALL BEHIND

We cut *nodes*, never phases. Losing the Drive node costs almost nothing. Losing phase 3 means every later node needs a hand-built form, and only one person can work at a time.

OUR WEEKLY RHYTHM

Monday: 30 minutes, agree this week's “done when”.
 Thursday: 30 minutes, everyone shows what actually runs.
 Nothing else. Two meetings a week is enough if both end in a demo.

Everything we will install, and when

This is the complete list for the semester. **Nobody needs to learn it all now** — each phase adds two or three things, and roughly half of them are already built into Node.js.

WHAT WE WRITE OURSELVES vs WHAT WE GET FOR FREE

We write this

the engine loop · the node shape · every node
the settings-form generator · expressions
webhook handling · the schedule registry
the Google permission flow · the sandbox
this is the project — the report writes itself

We do not write this

dragging and connecting boxes (React Flow)
the database (Postgres) · HTTPS (Caddy)
running on a schedule (croner)
the code editor box (Monaco)
solved problems — using them is a good decision

Figure 5. The rule for the whole project: build the thing the assignment is about, and borrow everything else.

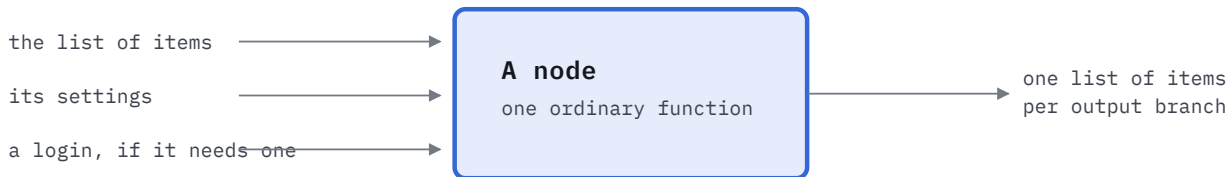
TOOL	WHAT IT IS FOR	INTRODUCED IN
Node.js 22	The language everything is written in. Already familiar from this semester.	Phase 1
Git + GitHub	One shared repository. Everyone works on a branch and merges.	Phase 1
TypeScript	Used lightly, to describe the node shape so mistakes appear in the editor rather than at runtime.	Phase 1
Docker Compose	Starts the database and the app together with one command, identically on all five laptops.	Phase 2
PostgreSQL 16	Stores workflows and every past run.	Phase 2
Express	The web server. Small enough to read in an afternoon.	Phase 2
React + Vite	The editor page in the browser.	Phase 2
React Flow	The canvas: dragging boxes, drawing lines, panning, zooming.	Phase 2
Monaco	The code-editing box in the sidebar — the same editor as VS Code.	Phase 3
croner	Runs a function on a schedule, with timezones handled properly.	Phase 4
Postman / curl	Sends test requests at our own webhook addresses.	Phase 4
cloudflared	Gives our laptop a temporary public web address while developing.	Phase 4
Google Cloud Console	Where we turn on the four Google APIs and set up the permission screen. Free.	Phase 5
node:crypto	Built into Node. Encrypts the stored Google keys.	Phase 5
worker_threads + vm	Both built into Node. How the Code node runs a user's JavaScript without taking us down.	Phase 5
Anthropic API	The AI node. One request, one response. Ollama can serve a local model instead.	Phase 5
AWSEC2	One small rented machine that runs the finished platform, about \$20 a month.	Phase 6
Caddy	Handles the HTTPS certificate automatically. Three lines of configuration.	Phase 6
n8n (Docker)	The platform we compare ourselves against, set up the same way as ours.	Phase 6
k6	Sends measured load at both platforms so the comparison uses real numbers.	Phase 6

A workflow runs, from a single file

What we are doing. Writing the loop from the previous page as one small program, and agreeing the exact shape every node will have. No web page, no database, no canvas — just a file we run and watch print results.

Why first. Everything else in the project plugs into this. If four of us start building nodes before this shape exists, we are all guessing, and we will all be wrong in different ways.

EVERY NODE WE EVER WRITE HAS THIS SHAPE



AND SO ALL OF THESE ARE THE SAME KIND OF THING



Figure 6. Agreeing this shape is the real work of phase 1. Once it is fixed, anyone can add a node without asking anyone else anything.

TOOLS WE USE IN THIS PHASE

Node.js	the language we already learned this semester — nothing new	VS Code + Git	one shared repository, everyone on the same branch rules
TypeScript	used lightly, only to describe the node shape so mistakes show up early	No frameworks	deliberately — this phase is plain JavaScript in one file

HOW WE PROCEED

1. **One person** writes a single file containing a hard-coded workflow, two node functions, and the loop. It should print real data from a weather API.
2. **All five of us sit down and read it together** until nobody has a question. This is worth an hour of everyone's time — it is the only part of the project that is genuinely conceptual.
3. **We agree the node shape** and write it down in the repository: what a node receives, what it must return, what it is never allowed to do.
4. **We write two boring nodes against it** — one that adds fields to each item, one that splits the flow in two. No Google, no APIs, nothing that can fail for outside reasons.
5. **We freeze the shape.** After this, changing it needs everyone to agree, because a change breaks work in progress.

DONE WHEN

A chain of three nodes runs from a script, the branch node sends data down the correct path, and every one of us can explain the loop without notes.

WATCH OUT

Do not put five people on this. One writes, four read. Parallel work on an unstable core produces five rewrites — this is the most common way a project like ours quietly fails in week three.

Workflows survive a restart, and we can draw them

What we are doing. Giving the engine a home. A database remembers workflows, a small web server hands them out, and a canvas in the browser lets us draw one instead of typing JSON by hand.

Why now. This is the point where the project stops being a script and starts being an application we can show someone. It is also the moment the group splits into parallel work for the first time.

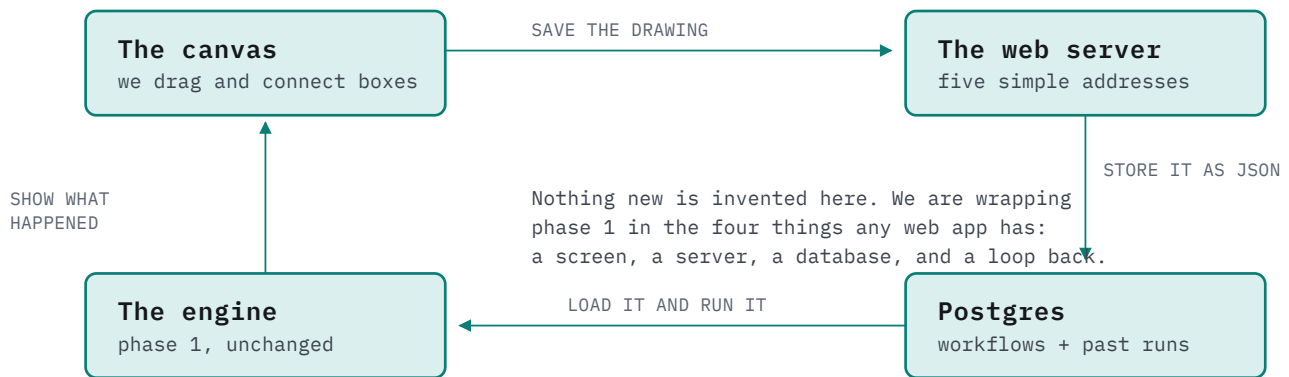


Figure 7. The full round trip. Once this circle closes, every later phase is just adding better boxes to the canvas.

TOOLS WE USE IN THIS PHASE

Docker Compose	starts the database with one command; nobody installs Postgres by hand	PostgreSQL	stores the workflows and every past run
Express	the smallest useful web server — readable end to end in an afternoon	node-postgres	talks to the database with plain SQL; no ORM to learn
React + Vite	the editor page itself	React Flow	dragging, connecting, panning and zooming — already solved for us

HOW WE PROCEED

1. **Start a database in Docker.** One config file, one command. Nobody installs Postgres on their laptop by hand.
2. **Two tables only.** One for workflows, one for runs. We store the whole drawing as a single blob of JSON rather than splitting it into rows — that is what n8n does too, and it keeps saving simple.
3. **A small web server with five addresses:** list workflows, get one, create one, save one, run one. That is genuinely enough for the next month.
4. **Build the canvas with React Flow.** Dragging, connecting, panning and zooming are solved problems — we do not write them ourselves. Our job is only the boxes and the sidebar.
5. **Join the two together:** the drawing saves to the database, and a Run button executes it and shows what each node produced.

DONE WHEN

We can drag out two nodes, connect them, refresh the browser and find them still there — then press Run and see the result of each one.

WATCH OUT

Converting between React Flow's format and ours is the one fiddly job in this phase, and essentially every canvas bug for the rest of the semester will live in that conversion. Write it once, carefully, with someone reading over your shoulder.

Nodes describe themselves, and fields can hold live data

What we are doing. Two things that look small and decide the rest of the project. First: a node *describes* its own settings, and one piece of code turns any description into the sidebar form. Second: any field can contain `{{ }}` and get filled in with real data before the node runs.

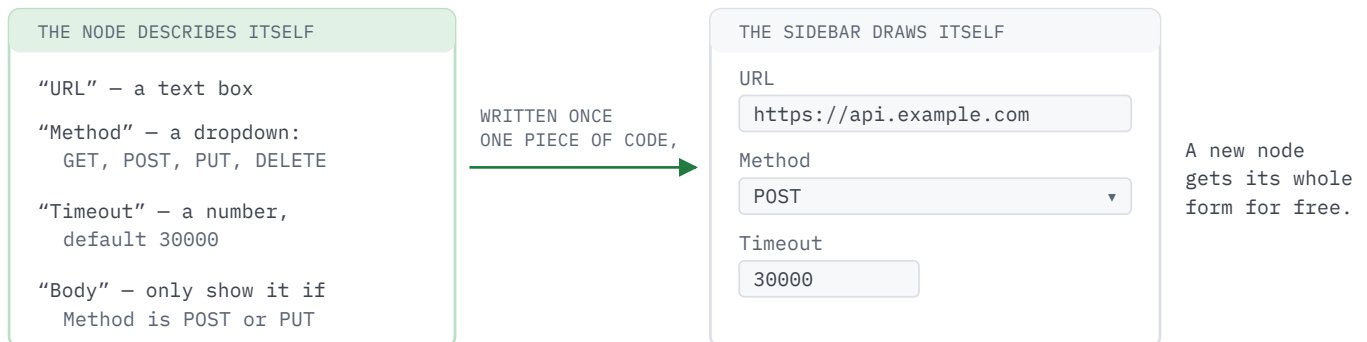
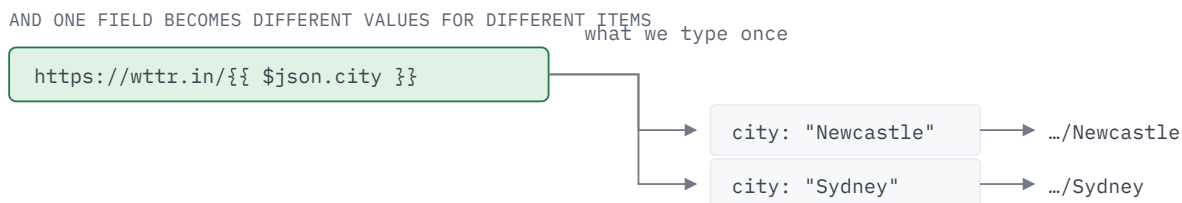


Figure 8. Nobody hand-builds a settings panel. Add a node file, restart, and its form is simply there.



TOOLS WE USE IN THIS PHASE

React	the one component that turns a settings description into a form	Our own code	no library does this for us — it is about a hundred lines, and it is the clever part of the project
Monaco	the editor from VS Code, dropped in later as the Code node's text box		

HOW WE PROCEED

1. **Agree the kinds of setting we support:** text, number, on/off, dropdown, a big code box. Five is plenty; more can be added later without breaking anything.
2. **Write the one component** that reads a description and produces those inputs.
3. **Add "only show this field when..."** so a form changes as you pick an operation — this is what makes the Google nodes feel finished rather than overwhelming.
4. **Add expressions** so any field can pull data from earlier nodes, filled in separately for each item before the node runs.

DONE WHEN

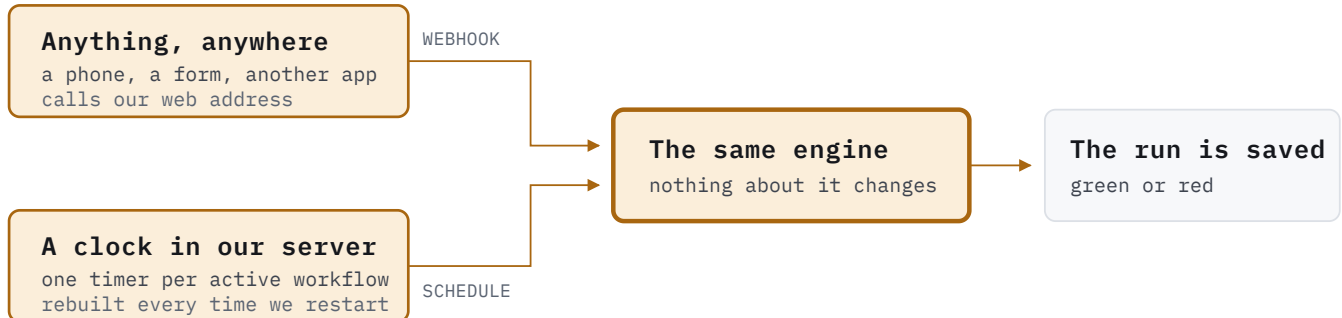
One of us writes a brand-new node file, restarts the server, and its complete settings form appears in the sidebar — with nobody having touched the editor.

WATCH OUT

This is the phase that is tempting to skip "to save time". Skip it and every one of the next ten nodes needs a hand-built form, only one person can work on the UI at a time, and weeks 8-10 turn into a queue behind them.

Workflows that run without anyone pressing a button

What we are doing. Giving workflows two ways to start on their own: a **web address** anything on the internet can call, and a **clock** inside our server. These are the two triggers the assignment asks for, and together they are what turns a tool into a platform.



Each workflow gets two addresses, exactly like n8n: a test one while we are editing it, and the real one once it is switched on.

Figure 9. Both routes end in the same loop. Adding a trigger never means changing the engine — that is the payoff for getting phase 1 right.

TOOLS WE USE IN THIS PHASE

croner	one small library that runs a function on a schedule	Postman or curl	to fire test requests at our own web addresses
cloudflared	a free tunnel that gives our laptop a public address while developing		

HOW WE PROCEED

1. **Give each workflow its own web address.** When a request arrives, we look up which workflow owns that address and start it, handing the request in as the first item.
2. **Two addresses per workflow** — a test one while editing, a live one once activated — so we never fire real emails while experimenting.
3. **Decide what the caller gets back:** an instant “received”, or wait for the workflow to finish and return its result. Both are useful; we support both.
4. **For schedules, keep one timer per active workflow** and rebuild the whole set whenever the server restarts — otherwise schedules silently stop after every deploy.
5. **Always store a timezone** with a schedule. Our laptops are on Australian time and the server will be on UTC.

DONE WHEN

A request sent from someone's phone runs a workflow on the laptop, and a scheduled workflow produces exactly one run per minute — stopping the moment it is switched off.

WATCH OUT

To test from outside while developing, use a tunnel tool rather than trying to open ports on the university network. It takes one command and saves an afternoon.

First, permission to use a Google account

What we are doing. Before any Google node can work, our server needs permission to act on someone's behalf — without ever seeing their password. Google hands us a set of keys instead. **This is the step most likely to cost us a week**, so we do it early and once.

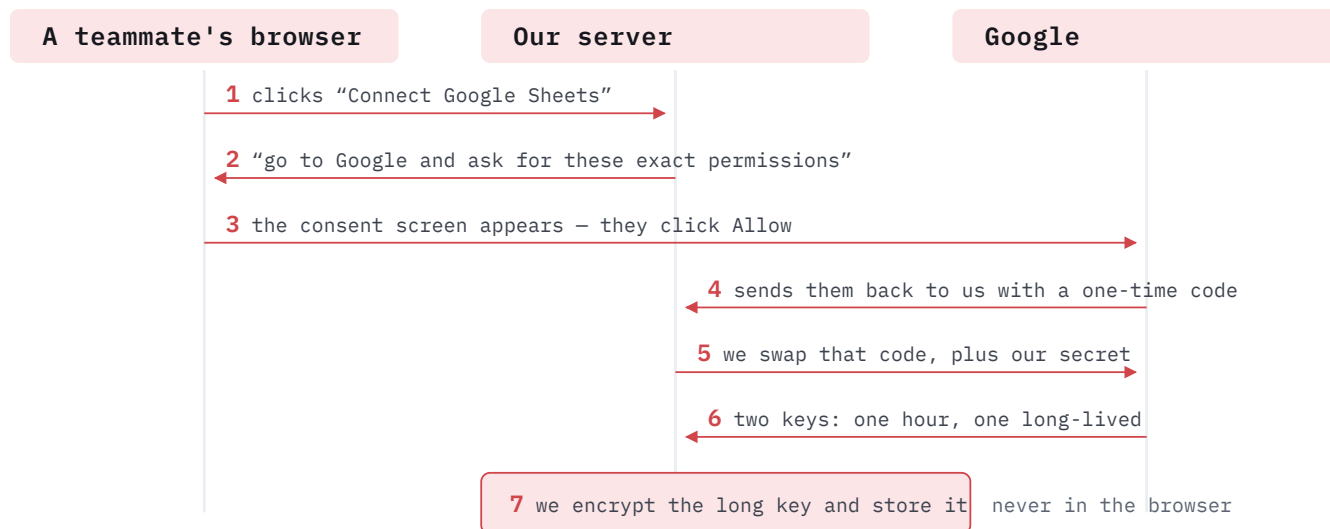


Figure 10. The whole “OAuth” process in seven arrows. Steps 1–3 happen in front of the user; 4–7 happen invisibly in about half a second.

TOOLS WE USE IN THIS PHASE

Google Cloud Console	where the project, the four APIs and the consent screen live — free	OAuth 2.0	the standard for “let this app act for me”. Not a library; a sequence of requests
node:crypto	built into Node — encrypts the stored keys, no package needed		

HOW WE PROCEED

1. **One of us creates a Google Cloud project** and switches on the four APIs we need: Sheets, Drive, Docs and Gmail. Free, about fifteen minutes.
2. **Fill in the consent screen** — the page users see when they click Allow — and keep the app in **Testing** mode. Then **add every group member as a “test user”**, plus our demonstrator's account: in Testing mode, nobody outside that list can connect at all.
3. **Build the redirect**: our server sends the user to Google, Google sends them back to a fixed address on our server with a one-time code, and we trade that code for the keys.
4. **Encrypt the long-lived key before storing it**, and never send it to the browser.

THE TWO TRAPS THAT END DEMOS

1 • Gmail permissions are “restricted”.

Publishing the app would need a Google security review we cannot finish this semester — and we do not need one. Testing mode plus a test-user list behaves identically for us.

2 • In Testing mode the long-lived key dies after seven days.

An account connected in Week 9 is disconnected by Week 14, and it looks like a broken node rather than an expired key. **Reconnecting every Google account is step one of the demo-morning checklist.**

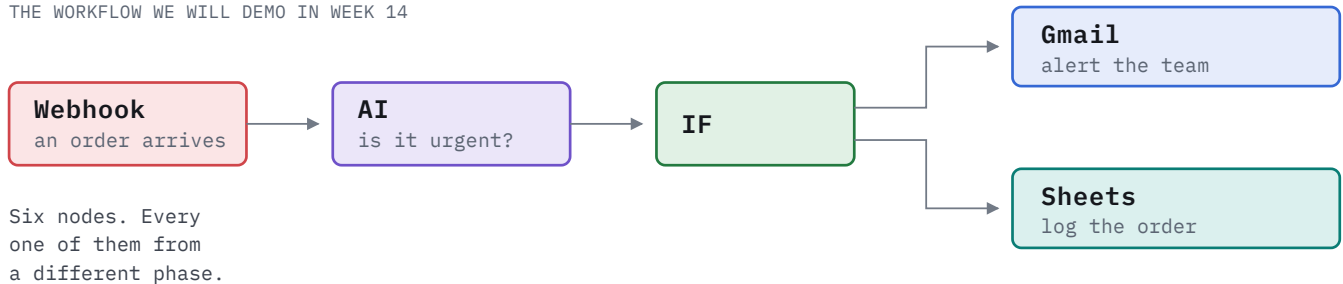
DONE WHEN

Someone clicks Connect, approves on Google's own screen, and our server can read a real spreadsheet using the stored keys.

Then the nodes people actually came for

What we are doing. Building the four Google nodes, the Code node, and the AI node. With phases 1-4 done, each of these is a small, self-contained job — which is exactly why four of us can work at the same time without getting in each other's way.

THE WORKFLOW WE WILL DEMO IN WEEK 14



Six nodes. Every one of them from a different phase.

Figure 11. Everything in the deck exists to make this one screen work in front of a demonstrator.

TOOLS WE USE IN THIS PHASE

Google REST APIs	Sheets, Drive, Docs and Gmail, called directly so we can explain them in the report	fetch	built into Node 22 — no HTTP library to install
worker_threads + vm	both built into Node; how we run user code without risking the server	Monaco	the code box in the sidebar
Anthropic API	the AI node — one request, one response	Ollama	optional: the same node pointed at a local model, as in our weather project

HOW WE PROCEED

1. **Build the Google nodes easiest first: Sheets, then Docs, then Drive, then Gmail.** Each one is a settings description plus a single call to Google. One shared helper deals with expired keys, so no node has to think about it.
2. **Allow a full day for Gmail.** Sending an email means assembling the message in an old format by hand — it is the only genuinely fiddly one, and knowing that in advance stops it feeling like failure.
3. **The Docs node is the best demo.** A template document with placeholders, filled in from a spreadsheet row — a mail merge in four nodes, and it looks impressive in ten seconds.
4. **The Code node lets users write JavaScript.** The work is not running it, it is stopping bad code from taking our server down — so we run it in a separate thread we can kill after five seconds.
5. **The AI node is one call and half a day.** Because settings fields can contain live data, someone can type “Classify this email: {{ ...the message... }}” straight into the sidebar and feed the answer into an IF node. Do not leave this until last — it is what makes ours an AI/automation platform.

THE CODE NODE: THREE WALLS, AND AN HONEST ADMISSION

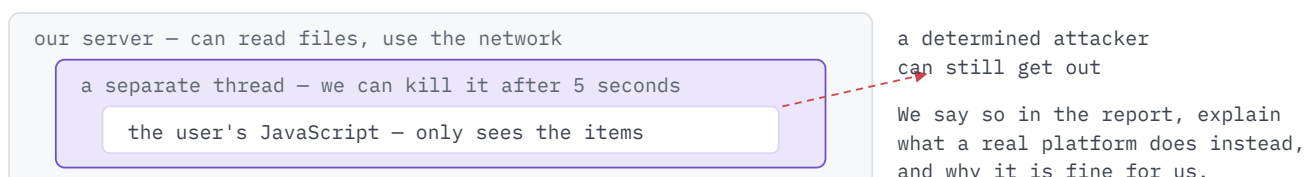


Figure 12. Being able to explain the limits of our own sandbox is worth more marks than quietly using a stronger one would have been.

DONE WHEN

The workflow in Figure 10 runs green from end to end, with a real email arriving and a real spreadsheet row appearing.

Online, measured, and written up

What we are doing. Putting the platform on a real server so our webhook addresses work from anywhere, then measuring it honestly against n8n, then writing the report. **At the end of Week 11 we stop adding features** — everything after that is fixing, measuring and writing.

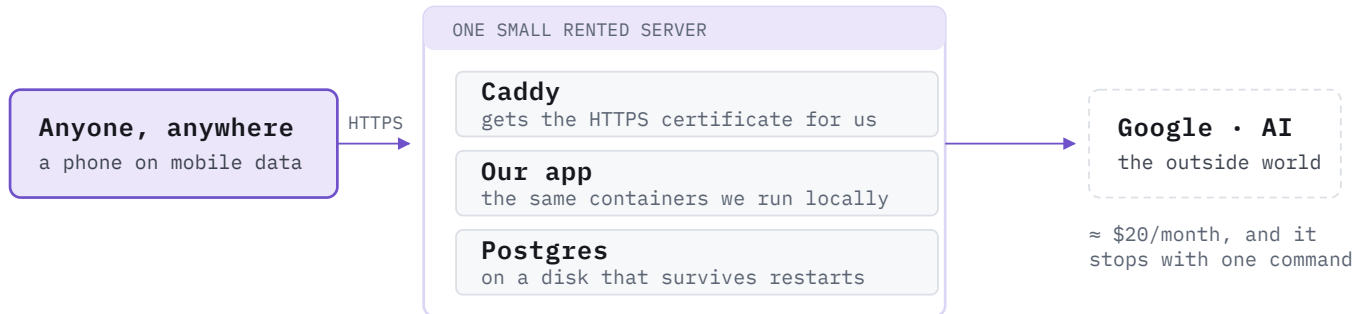


Figure 13. Not a cloud architecture diagram — one machine running the same three containers we already use. An afternoon's work, not two weeks.

TOOLS WE USE IN THIS PHASE

AWS EC2	one small rented machine — the whole platform runs on it	Caddy	gets and renews the HTTPS certificate by itself; three lines of configuration
n8n Docker image	the thing we are comparing against, set up the same way as ours	k6	sends measured load at both platforms so the comparison is real
GitHub Actions	runs our checks automatically on every change		

HOW WE PROCEED

1. **Freeze features at the end of Week 11.** Whatever is not working by then does not go in. This is a group decision made in advance so it is not an argument later.
2. **Rent one small server, point a cheap domain at it,** and let Caddy handle HTTPS. Set a spending alarm on the first day, before launching anything.
3. **Run n8n on the same machine, configured the same way** — not its simplest setup, or the comparison is unfair and a marker will notice.
4. **Build the same three workflows on both,** run each three times, and record the numbers with the raw data kept.
5. **Time ourselves adding a new node to each platform.** We will win this one clearly, and nobody else will think to measure it.
6. **Write honestly.** n8n does far more per run than we do, so being faster mostly means “we do less”. Saying that plainly is worth more marks than the number itself.

DONE WHEN

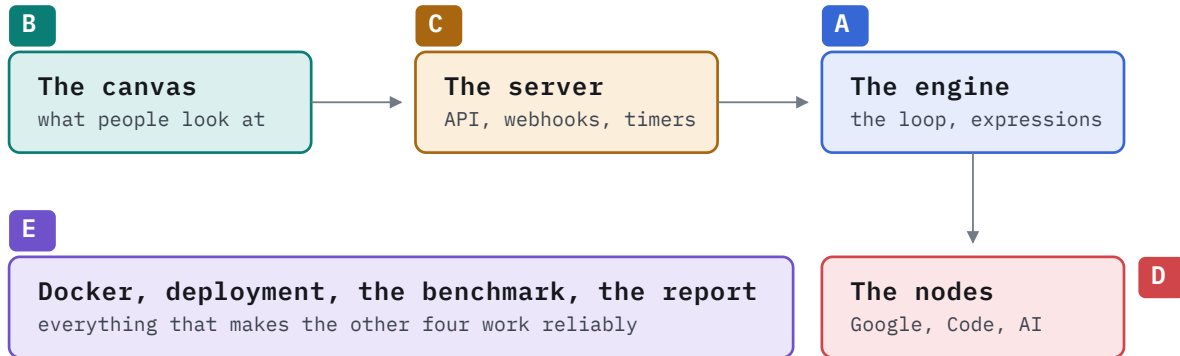
The editor and a webhook address both work from a phone on mobile data, the benchmark numbers are recorded, and the report draft is circulating.

WATCH OUT

Write the report from Week 5 onwards, a section at a time, and screenshot everything as we build it. You cannot re-screenshot a server you have already shut down.

Who owns what

We own **parts of the system**, not tasks. That way nobody has to ask whose decision something is, and we rarely edit the same file at the same time.



Two agreements hold it together: the node shape (frozen in phase 1) and the list of server addresses (frozen in phase 2).

Figure 14. With six people, D splits in two: Sheets and Drive on one side, Gmail, Docs and AI on the other.

STREAM A · ENGINE

Owns the loop, the item model and expressions. Writes phase 1 alone, then supports everyone else. This is the technical-lead seat.

STREAM B · CANVAS

Owns everything in the browser: the drawing surface, the settings panel generator, the run viewer. The largest single piece of work.

STREAM C · SERVER

Owns the API, webhooks, the schedule timers, and the Google permission flow. The trickiest debugging in the project lands here.

STREAM D · NODES

Owns every integration. Cannot start properly until phase 3 is finished — before that, writes specifications and does the Google Cloud setup.

STREAM E · INFRASTRUCTURE, TESTING AND THE REPORT

Owns Docker, deployment, the benchmark harness, and is editor-in-chief of the report and director of the demo. Starts on day one, not in Week 12 — the report is written a section a week from Week 5.

Three things that end projects like ours

Google keys expire after seven days

While our app is in Testing mode, the long-lived key stops working after a week. Connected in Week 9, dead by Week 14 — and it looks like a broken node, not an expired key.

What we do: reconnect every Google account on the morning of the demo. It is step one of the checklist.

Gmail permissions are restricted

Making the app public needs a Google security review we cannot complete this semester.

What we do: stay in Testing mode and add every member — plus our demonstrator — as test users in Week 7, not in Week 8 when we first need it.

Nothing works on demo day

Campus wifi, an API outage, a laptop that decides to update. It happens, always at 9am.

What we do: record a full video of the working demo in Week 13, bring a second laptop, and have a phone hotspot ready.

Six habits that will save us weeks

- 01 **Build the boring version first.** An HTTP node that only does the simplest request is worth more in Week 6 than a perfect one in Week 9, because everything downstream can start.
- 02 **One person owns the engine.** Two people editing it at once produces a mess nobody can untangle.
- 03 **We are not building user accounts.** Logins and password resets earn no marks here and will eat a week.
- 04 **When a node will not work, test the plain request in Postman first.** Most “my node is broken” turns out to be a missing permission or a wrong header — nothing to do with our code.
- 05 **Print the data between nodes, constantly.** Almost every confusing bug is the shape of the data, not the logic.
- 06 **Copy n8n's good ideas deliberately, and say so in the report.** Explaining *why* a design is right is exactly what the assignment is asking for. Copying without understanding is what costs marks.

What we each do this week

WHO	TASK
A	Write the engine script from Figure 3, then walk the whole group through it. Everything else waits on this.
B	Get a React Flow canvas running that can add a box and connect two boxes. Nothing needs to save yet.
C	Postgres in Docker, the two tables, and a web server that answers a health check.
D	Create the Google Cloud project , switch on the four APIs, fill in the consent screen, add everyone as test users. This week — not in Week 8.
E	Repository set up, the shared Docker file, automatic checks on every change, and an AWS account with a spending alarm.
All	Put the Week 13 rehearsal in the calendar now, while it is still easy to schedule.

ONE LAST THING TO SAY OUT LOUD

If a phase takes twice as long as this deck says, that is completely normal. We cut a node, never a phase — and we never start a new phase before the previous one's “done when” is actually true.

How six phases become one working system

What we are doing. Each phase produced a piece, and each of us has been working in our own folder. Integration is not a week at the end — it is a habit from Week 4: **one repository, one command, and everything runs.** If assembling the project is ever difficult, something has gone wrong earlier.

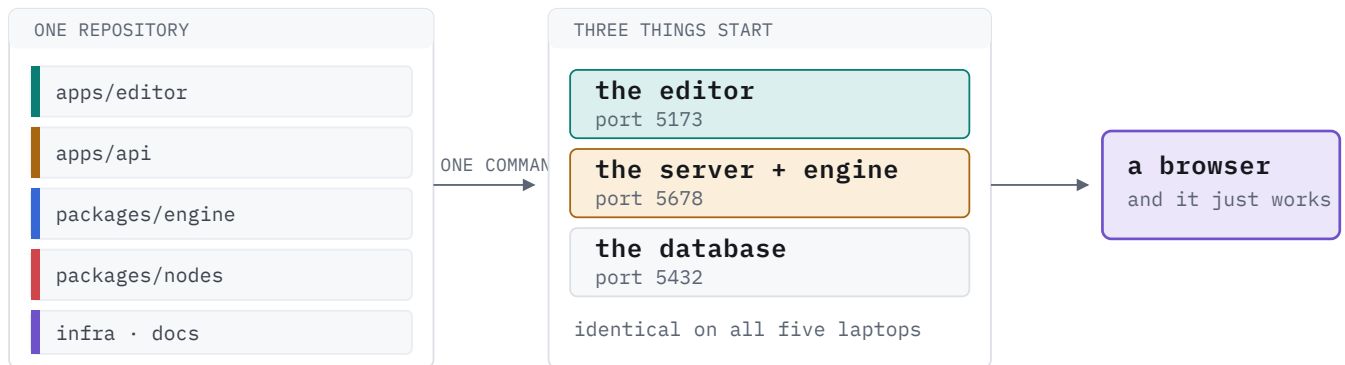


Figure 15. Five folders, one command, three processes. Anyone who can clone the repository can run the whole platform.

HOW WE PROCEED

1. **One repository from day one**, with everyone on their own branch and merging into main. Never a zip file passed around, and never “I will send you my folder”.
2. **One command starts everything.** `docker compose up` brings up the database, the server and the editor together. If that command ever stops working, fixing it is more urgent than any feature.
3. **One example settings file** in the repository listing every key the project needs, with fake values. Real keys live only in each person's own untracked copy.
4. **A starter workflow loaded on first run**, so a fresh clone shows something working instead of an empty screen. This is the difference between a new teammate being productive in ten minutes or in two days.
5. **Integration every Thursday.** Everything merges to main, and one person — a different one each week — clones the repository fresh and runs it. Whatever breaks is that week's priority.

DONE WHEN

Someone who has never touched the project can clone it, run one command, and have a working platform in under ten minutes without asking a question.

WATCH OUT

The classic failure is five people whose parts each work alone and have never run together. Merging weekly from Week 4 makes integration boring — which is exactly what we want it to be.

Four levels of testing, and what each one catches

What we are doing. Not testing everything — testing the things that break quietly. The rule: **the lower the level, the more tests and the faster they run.** We do not need all four levels from day one; we add each one as the phase that needs it arrives.

1 • Engine tests

about 20 • milliseconds

Does the loop run nodes in the right order? Does a branch with zero items correctly stop everything after it?

2 • Node tests

one per node • seconds

Does the Gmail node build the right request from its settings? Runs against a fake server, never real Google.

3 • Browser test

one or two • a minute

Can a person drag out two nodes, connect them, press Run, and see the result? One test covering the whole path.

4 • The real run

by hand • before demos

Does a real email actually arrive? Does a row appear in a real spreadsheet? Only a human can confirm this.

Levels 1–3 run automatically on every change. Level 4 is a checklist we work through before every demo.

Figure 16. Each level catches a kind of bug the level above it cannot. Skipping level 2 is why integrations break silently the week before a demo.

TOOLS WE USE FOR TESTING

Vitest

runs levels 1 and 2 — fast, and the default for a Node project

A fake server

a tiny local server that answers like Google does, so node tests never hit the real thing

Playwright

drives a real browser for level 3 — already installed for the demo screenshots

GitHub Actions

runs levels 1–3 automatically on every change, so nobody has to remember

HOW WE PROCEED

1. **Start with the engine (level 1), in phase 1.** These are the cheapest tests we will ever write and they protect the piece everything else depends on.
2. **Add one test per node as we build it (level 2),** not afterwards. The trick is the fake server: our node calls a local address instead of Google's, so the test is fast, free, and works with no internet.
3. **Write exactly one browser test (level 3)** once the canvas can save. One is enough — browser tests are slow and break for boring reasons.
4. **Keep level 4 as a written checklist,** not as tests. The acceptance list on the next page is that checklist.
5. **Turn on automatic checks early,** so that from Week 5 onward a broken change is caught when it is pushed rather than the night before the demo.

DONE WHEN

Someone pushes a change that breaks the IF node, and a red cross appears within two minutes — without anyone having noticed by hand.

DO NOT TEST

Google's APIs, React Flow, or Postgres. They are not our code and testing them wastes time we do not have. Test *our* logic, and use a fake for everything else.

Proving the whole thing works, on the day

What we are doing. The last two weeks are not for building. They are for making sure that what we built works in front of someone else, on a day we do not control.

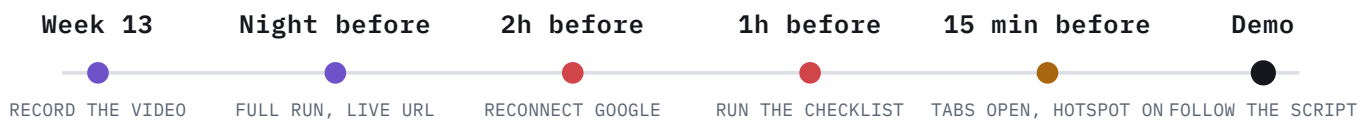


Figure 17. The two red markers are the ones that have ended other groups' demos. Neither takes more than fifteen minutes.

THE ACCEPTANCE CHECKLIST – EVERYTHING THE BRIEF ASKS FOR, AND HOW WE PROVE IT

WHAT IS REQUIRED	WHAT WE SHOW
Schedule trigger	A workflow set to every minute; the run list fills up while we talk, and stops when we switch it off.
Webhook	Someone in the room sends a request from their phone; the run appears on screen immediately.
HTTP request	The demo workflow calls a real outside API and shows what came back.
Google Sheets	A row appears in a spreadsheet that is open on screen.
Google Docs	A template document comes back with its placeholders filled in from that row.
Google Drive	That finished document appears in a Drive folder.
Gmail	A real email arrives in someone's inbox, with the document attached.
Code node	A snippet turns one item into ten; then an infinite loop is killed after five seconds and the server stays up.
AI node	The model classifies an incoming message and the IF node routes it to the right branch.
Extensibility	A brand-new node is written live in about ninety seconds and appears in the palette with a full settings form.
Comparison with n8n	One slide: our medians beside n8n's, the raw data available, and an honest sentence about why we are faster.

DONE WHEN

We have run this entire checklist twice, on the deployed URL, on two different machines — and the fallback video exists.

THE LAST THING TO SAY

Every row above maps to a sentence in the brief. If we can tick all eleven, the demo is finished — everything else is polish, and polish is not what is being marked.